THE HONG KONG POLYTECHNIC UNIVERSITY

IncidentLens

Evidence-driven multi-agent incident investigation

COMP5584: Large Language Models for Agents

Final Group Project, Group 5

YANG Ziting 25050315G

Song Xiaowen 25058859G

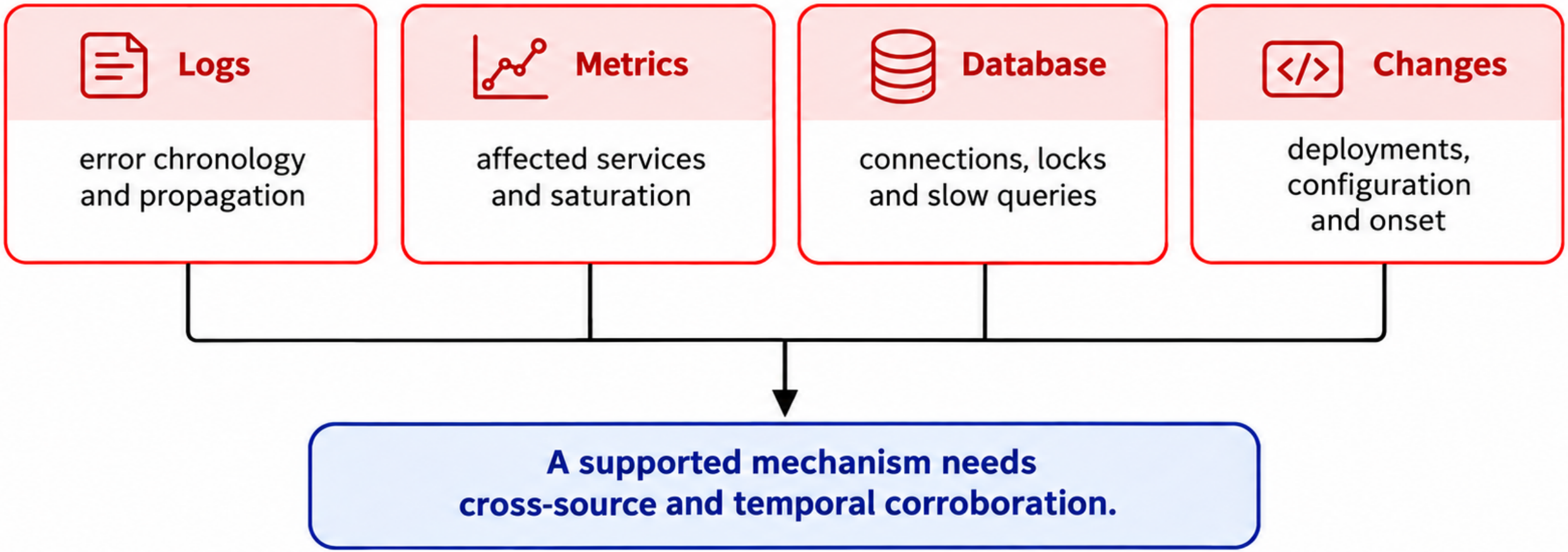
YE Mingxi 25065994G

MA Jiyuan 25116696G

Zhang Boxin 25054717G

WANG Huixuechun 25064113G

Incident diagnosis requires evidence across sources



Can coordination improve diagnosis within a fixed budget?

Research Question

Evidence specialization

Specialists investigate source-specific questions with bounded tools.

→

Shared investigation state

Merged evidence and stable hypotheses maintained and refined.

→

Measured outcomes

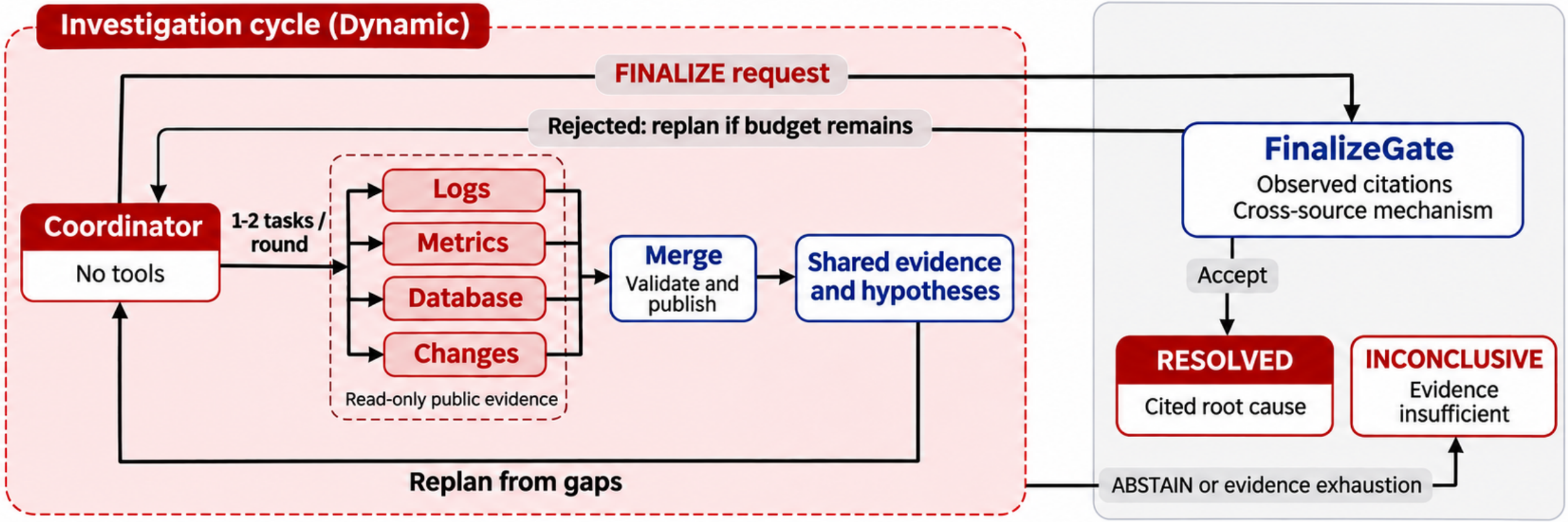
An auditable diagnosis (RESOLVED) or an appropriate abstention (INCONCLUSIVE).

- A coordinator allocates tasks from the current evidence gap.
- Compare quality and resource use across four matched architectures.

Synthetic development cases; read-only investigation; no production changes.
Multi-agent specialization and coordination are the design rationale, not proof of a universal advantage.

Dynamic coordinates specialists through shared evidence

A bounded investigation loop with deterministic acceptance.



Dynamic is the primary architecture. An independent verifier is an ablation.
Monitoring and PLAN_ONLY response are optional workflow extensions.

Model-driven role
Deterministic component

Each specialist has a bounded tool scope

Role	Main question	Read-only tools
Coordinator	What next?	No evidence tools
Logs	What failed first?	query_logs
Metrics	What is saturated?	query_metrics, get_dependencies
Database	Is the database exhausted or blocked?	get_db_*
Changes	What changed near onset?	query_changes, get_config_diff
Verifier	Does published evidence support the candidate?	fetch evidence by ID

- **Coordinator:** decide tasks and next steps; no evidence tools.
- **Logs:** query_logs. **Metrics:** query_metrics and get_dependencies.
- **Database:** read-only get_db_* tools. **Changes:** query_changes and get_config_diff.
- **Verifier:** fetch already-published evidence by ID.
- A task can narrow role authority. **Single** receives the investigator-tool union.

Equivalent-tool benchmark configuration may grant database access to Metrics. Role ceilings and task scopes remain enforced.

Tasks and findings update one shared investigation state

- Task: role, scope, question and expected discriminator.
- Finding: observed record IDs and hypothesis updates.
- Merge is the only publication boundary for shared findings.
- Stable hypothesis IDs retain evidence across rounds.
- Completed scopes, duplicate-task checks and budgets bound the loop.

Task packet example

Role	Scoped specialist
Scope	Bounded area of investigation
Question	Concrete question to answer
Expected discriminator	Evidence or signal that differentiates hypotheses

Investigation cycle (4 steps)

1. **Read** — Read shared state and status
2. **Assign** — Select and issue a typed task
3. **Investigate** — Specialist returns a finding
4. **Merge** — Validate and publish to shared state

Specialist response example

Role	Scoped specialist
Observed record IDs	IDs of relevant records
Hypothesis updates	Add / support / weaken (hypothesis ID, brief note)

Shared investigation state

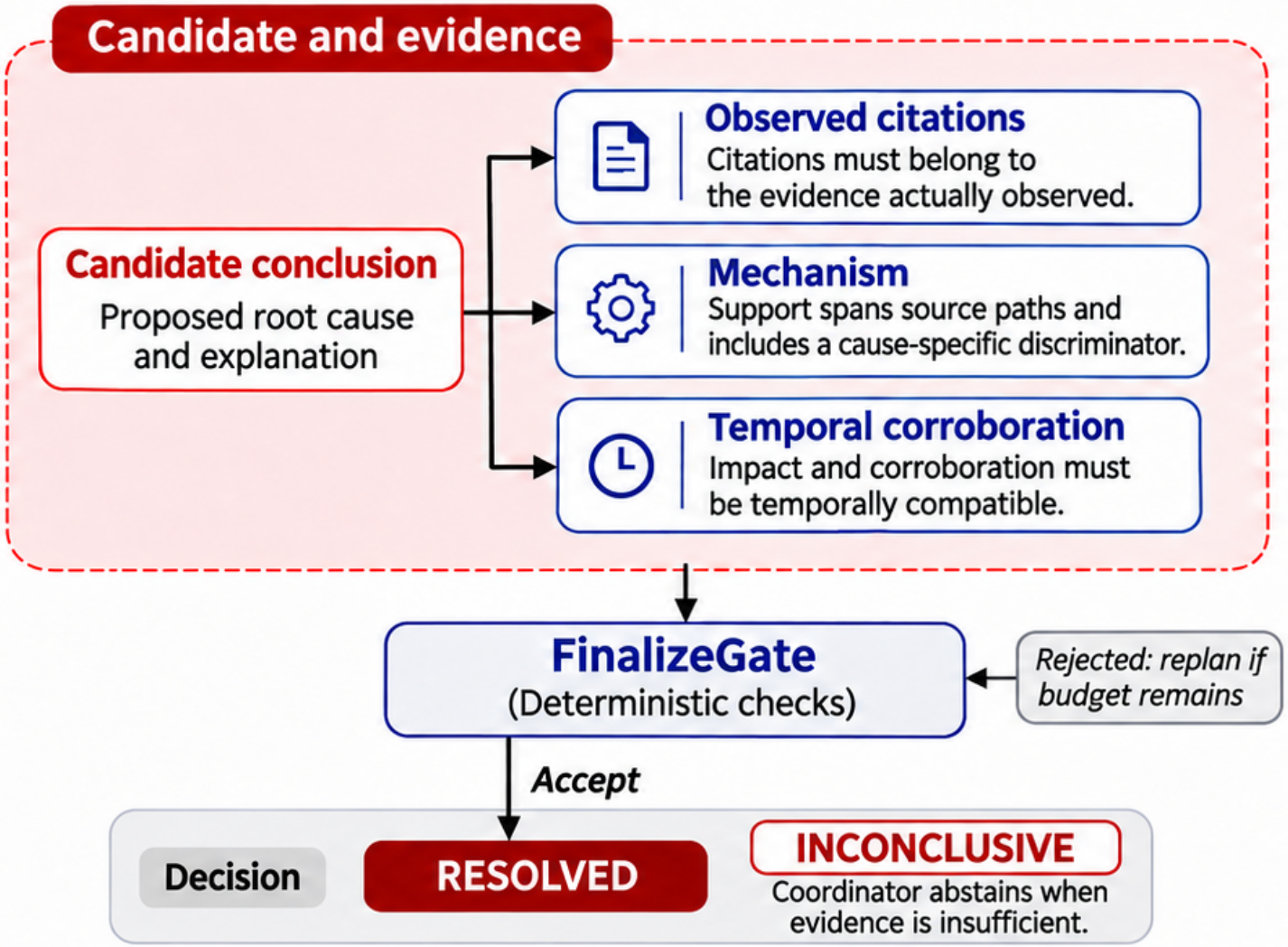
Shared evidence (read-only)
Published findings (record IDs, source, summary)

Hypotheses (stable IDs)
Hypothesis ID, status and supporting evidence

Packet fields are structural examples, not a real trace. Specialists return local proposals, not direct shared-state writes. The board is not an LLM agent.

Deterministic checks control conclusion acceptance

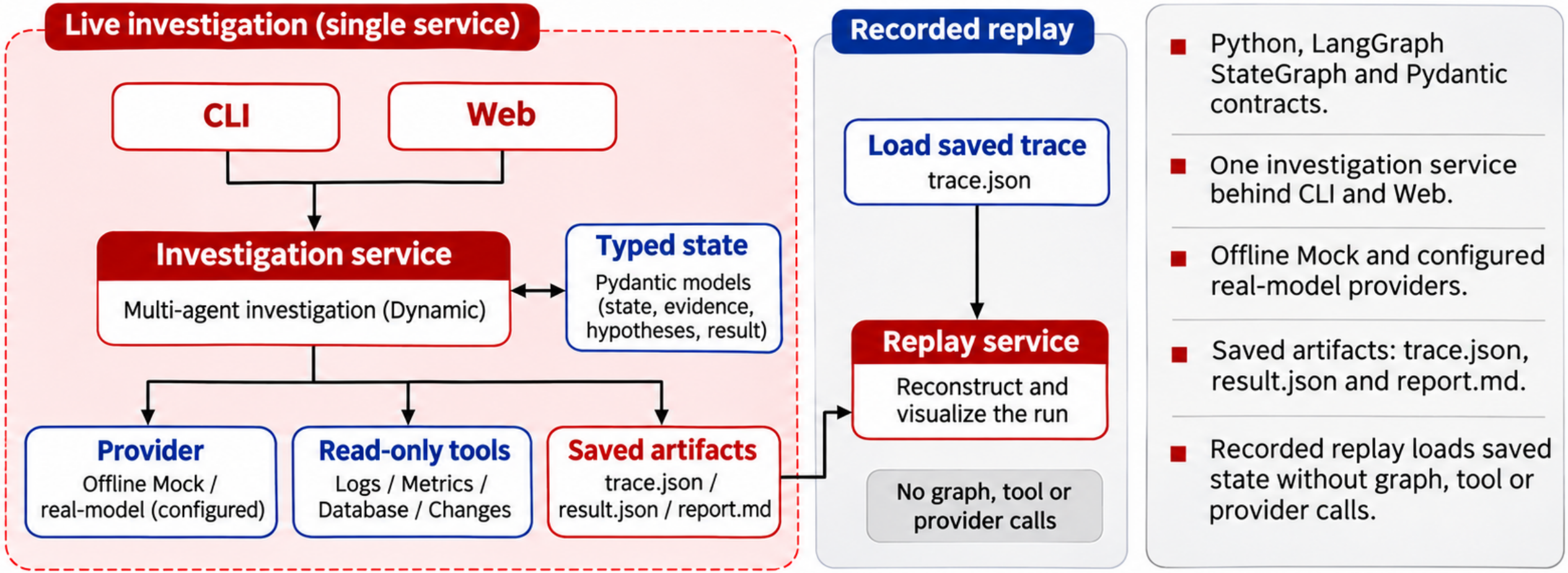
Deterministic evidence checks determine whether a candidate conclusion can be accepted.



1. Citations must belong to the evidence actually observed.
2. Support spans source paths and includes a cause-specific discriminator.
3. Impact and corroboration must be temporally compatible.
4. Contradictions and observed competing mechanisms can block acceptance.
5. Acceptance yields RESOLVED. Insufficient evidence can yield INCONCLUSIVE.

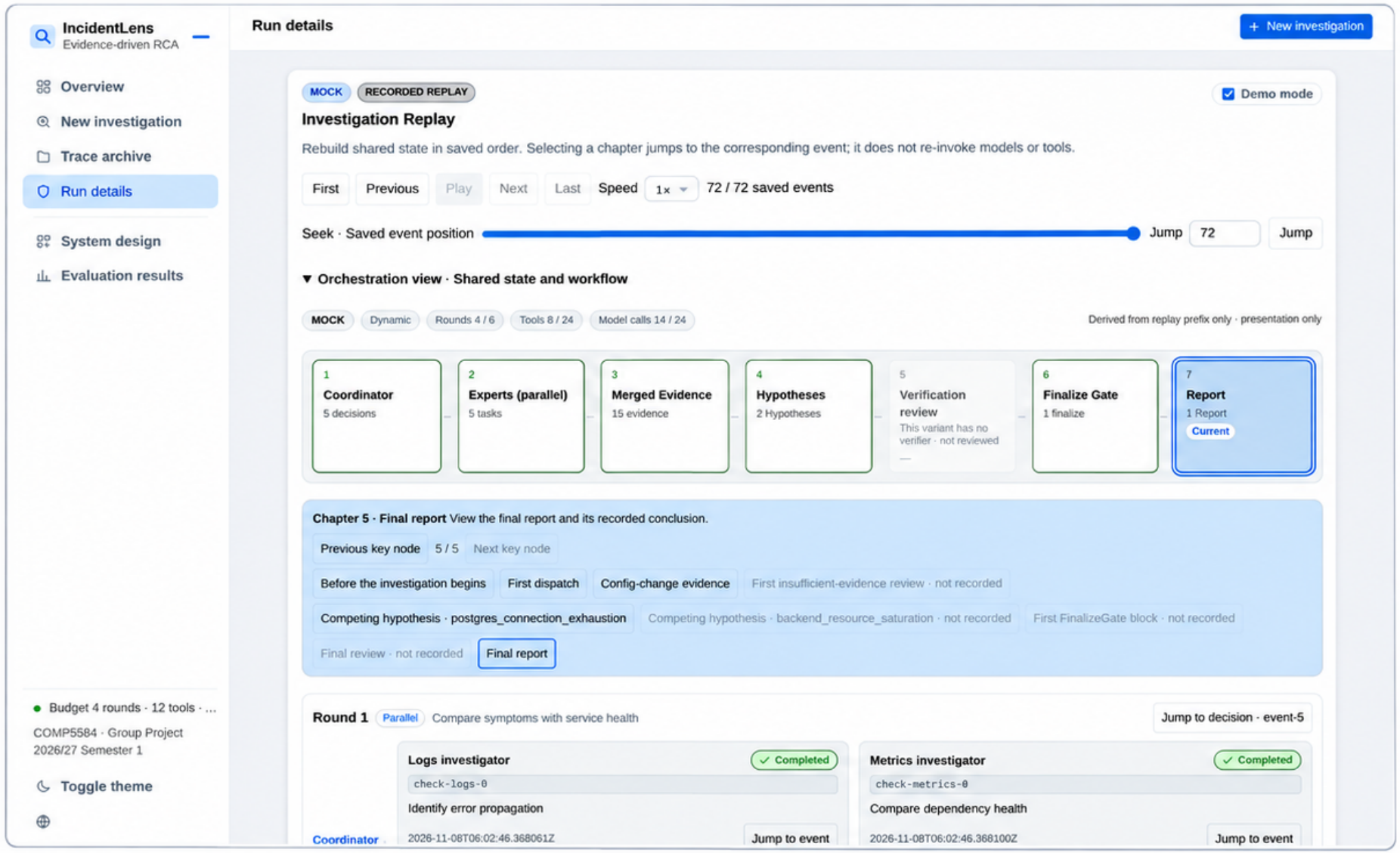
Structural provenance checks do not establish human-verified semantic support. No confidence score decides acceptance.

One implementation supports live runs and recorded replay



Note: The real-model provider is auto/deepseek-v4.1-flash through a local gateway.
All tools are read-only; no production integrations or automatic remediation execution.

The dashboard exposes decisions, evidence and outcomes



- Inspect task assignments and observed evidence.
- Follow hypothesis updates and the final gate decision.
- Open the cited report or explain missing evidence.

Shown example: Mock provider, recorded replay.

Four architectures share one evaluation protocol

Single; Fixed-4; Dynamic; Dynamic with verifier.

Architecture	Investigation schedule	Review
Single	One identity with all investigator tools	Self-review by the same identity
Fixed-4	One fixed four-specialist batch	No independent verifier
Dynamic	Adaptive selection, 1-2 tasks per round	No independent verifier
Dynamic with verifier	Adaptive selection, 1-2 tasks per round	Independent verifier ablation

Experimental design

36 synthetic development cases
29 identifiable, 7 requiring abstention

×

Four variants
Single / Fixed-4 / Dynamic / Dynamic with verifier

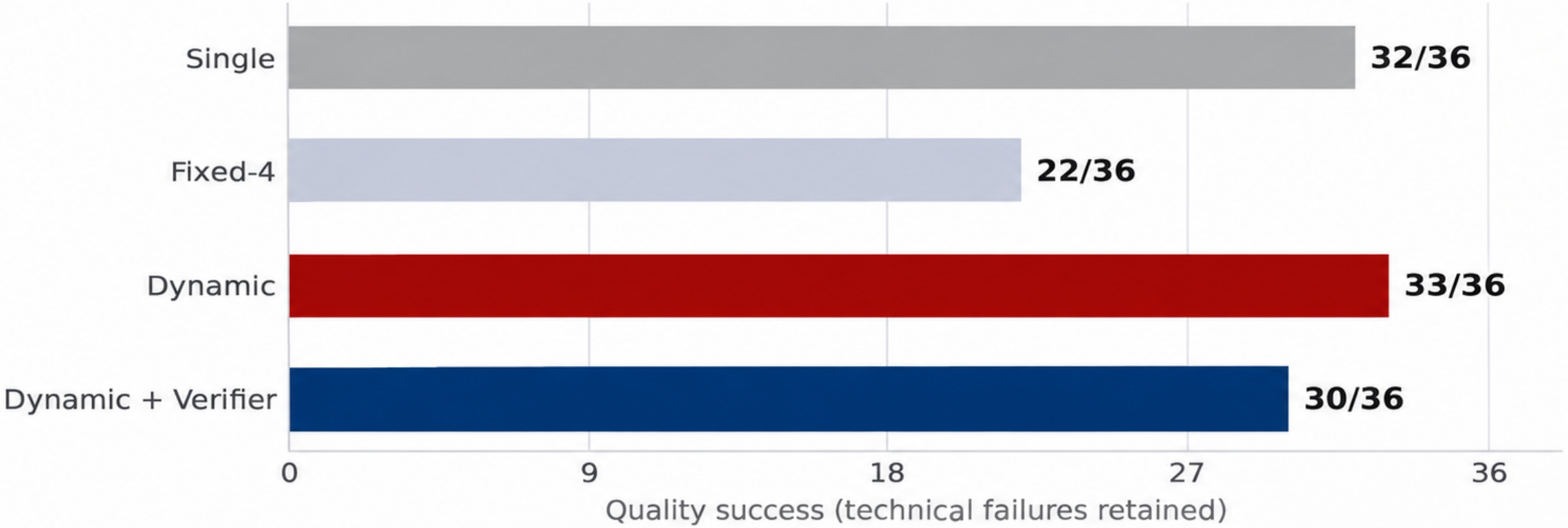
=

144 slots
One model and one repeat

- Matched public evidence, configured tools, budgets and model settings.
- **Quality success retains technical failures in the denominator.**

Recorded model: auto/deepseek-v4.1-flash via local gateway. Labels provisional; development-visible. Single self-review uses one identity. Fixed-4 uses one four-specialist batch. Dynamic adapts tasks per round. Verifier is a separate ablation of Dynamic.

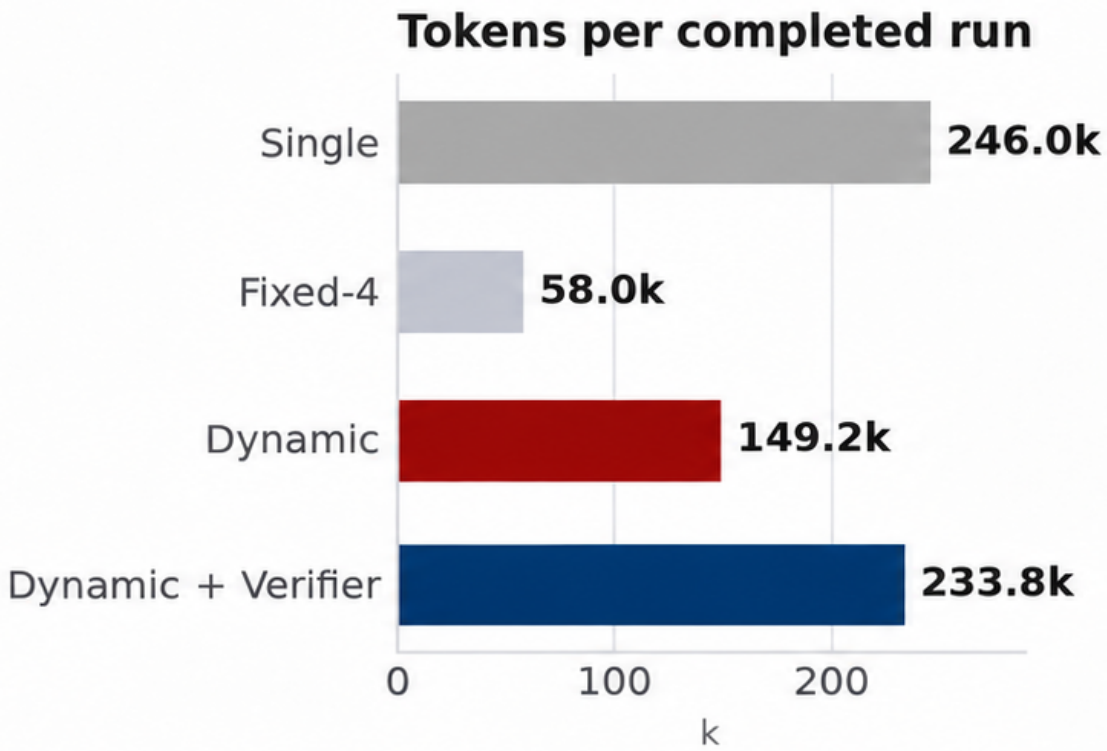
Dynamic achieved 33 quality successes out of 36 slots

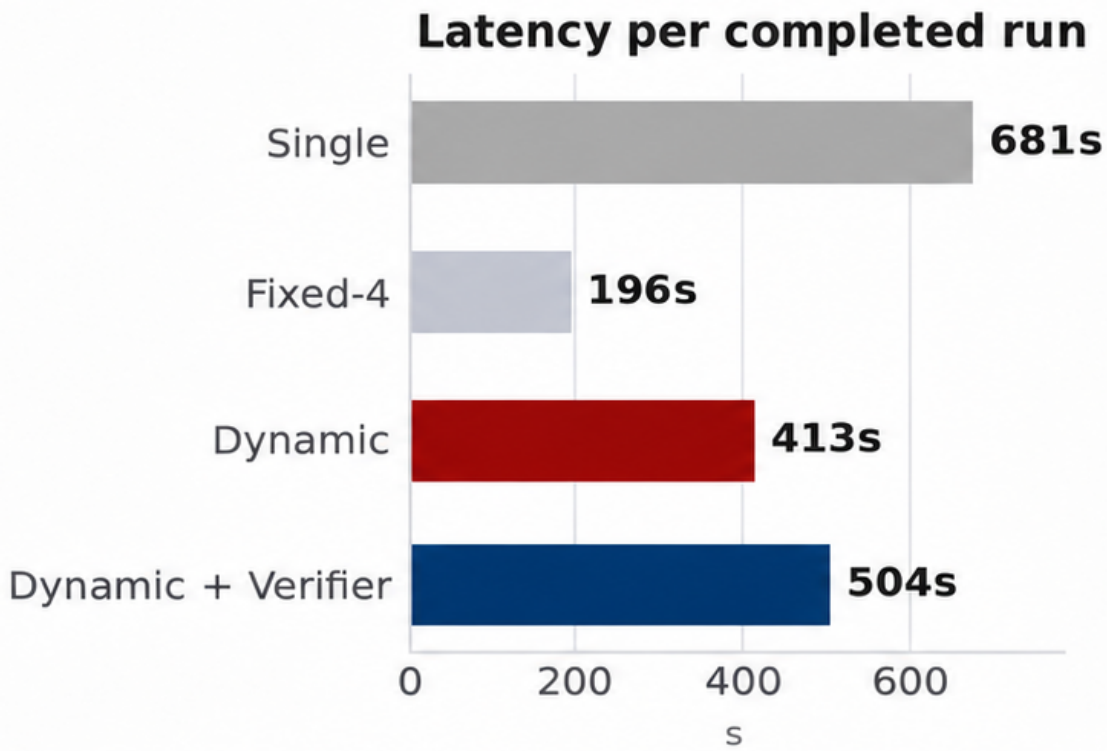


- **Dynamic: 27 correct roots + 6 correct abstentions = 33 quality successes.**
- Three technical failures stay in its 36-slot denominator.
- 27/27 root correctness is conditional on completed identifiable cases.

Development-visible synthetic batch; provisional labels; one model, one repeat.

Dynamic used fewer tokens than Single in this batch





Dynamic vs Single: 39.3% fewer tokens and 39.4% less latency.

Verifier vs Dynamic: 56.7% more tokens, with no observed quality gain.

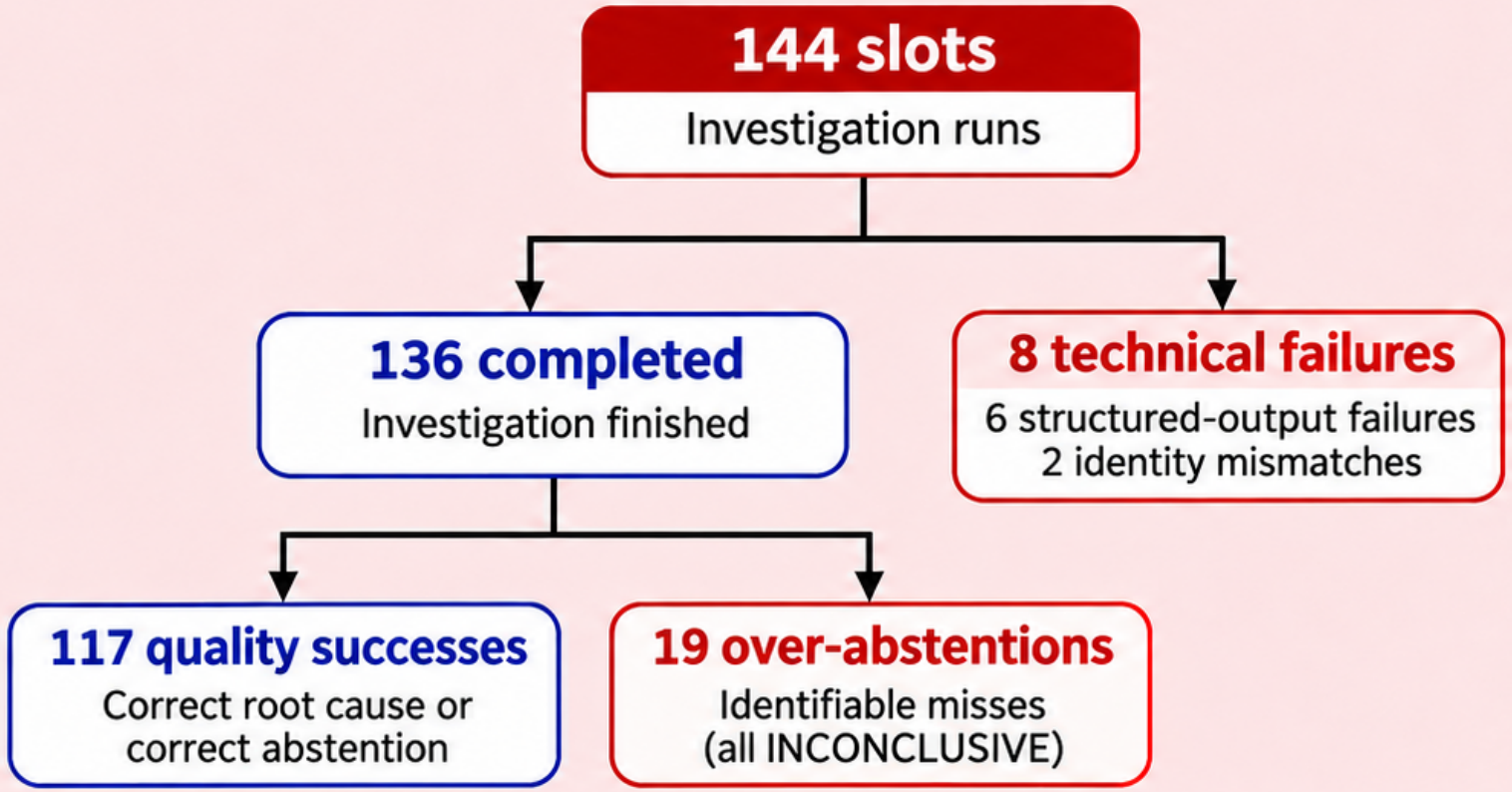
- Averages cover technically completed runs.

Descriptive differences; one model, one repeat; no significance or USD-cost claim.

Identifiable misses ended in INCONCLUSIVE

A labeled outcome decomposition of 144 slots and the main failure observation.

Outcome decomposition



Key points

- 144 slots: 136 completed and 8 technical failures.
- Completed runs: 117 quality successes and 19 identifiable misses.
- All 19 identifiable misses ended INCONCLUSIVE.
- The scorer found 0 wrongly named root causes in this batch.
- Observed quality failure: over-abstention reduces useful coverage.

Candidate labels are provisional. Technical failures are separate from over-abstentions (wrong abstentions).

The evidence supports a bounded development study

Study boundary (this study)	Next validation step
Synthetic, development-visible cases.	Evaluate with realistic telemetry (e.g., anonymized or internal incident data).
Candidate labels pending independent review.	Obtain independent labels and reconcile differences (e.g., via an adjudication process).
One model and one repeat.	Run repeated experiments and include multiple models (e.g., additional versions or families).
No blind test, human semantic audit or significance test.	Add blind evaluation, a human semantic audit (as needed) and statistical tests to assess differences.

Next validation: independent labels, repeated/multi-model runs and realistic telemetry.

A bounded development study of a research/teaching prototype (IncidentLens), not a claim of production performance.

Evidence discipline makes incident investigations auditable

1. Bounded roles and shared state make the collaboration inspectable.
2. Deterministic gates define when a root cause can be accepted.
3. Dynamic had the highest quality success count in this development batch.

Next priorities | Independent evaluation and better identifiable-case coverage.

Thank you. Questions?

Note: Evidence is based on 36 synthetic cases and one real-model repeat (144 slots, four variants) in a development setting; differences are descriptive without a blind test, significance claim, human semantic audit or production generalization.

Backup: Decision semantics and termination

Coordinator choices (decision semantics)	Meaning and runtime handling
INVESTIGATE	Assign scoped specialist work.
FINALIZE	Request checks on a selected hypothesis. Accepted: yield a cited root cause. Rejected: replan (if budget remains).
ABSTAIN	Stop with insufficient evidence.

Report statuses (investigation outcome)	Definition
RESOLVED	Requires an accepted cited root cause.
INCONCLUSIVE	May carry an unconfirmed leading explanation.

Key points

- **INVESTIGATE:** assign scoped specialist work.
- **FINALIZE:** request checks on a selected hypothesis.
- **ABSTAIN:** stop with insufficient evidence.
- **RESOLVED** requires an accepted cited root cause.
- **INCONCLUSIVE** may carry an unconfirmed leading explanation.

Termination categories

Evidence insufficiency is normal abstention; **BUDGET_EXHAUSTED** and unrecoverable execution errors are runtime failures. All terminal paths pass through stop before reporting.

A provisional explanation must never be labeled confirmed root cause.

Backup: Metrics use different eligible denominators

- Quality and technical completion use all 36 slots per architecture.
- Root correctness uses completed identifiable cases.
- Correct abstention uses completed abstention cases.

Architecture	Technical	Root correct	Abstain	Quality
Single	35/36	25/28	7/7	32/36
Fixed-4	34/36	15/27	7/7	22/36
Dynamic	33/36	27/27	6/6	33/36
Dynamic with verifier	34/36	23/27	7/7	30/36

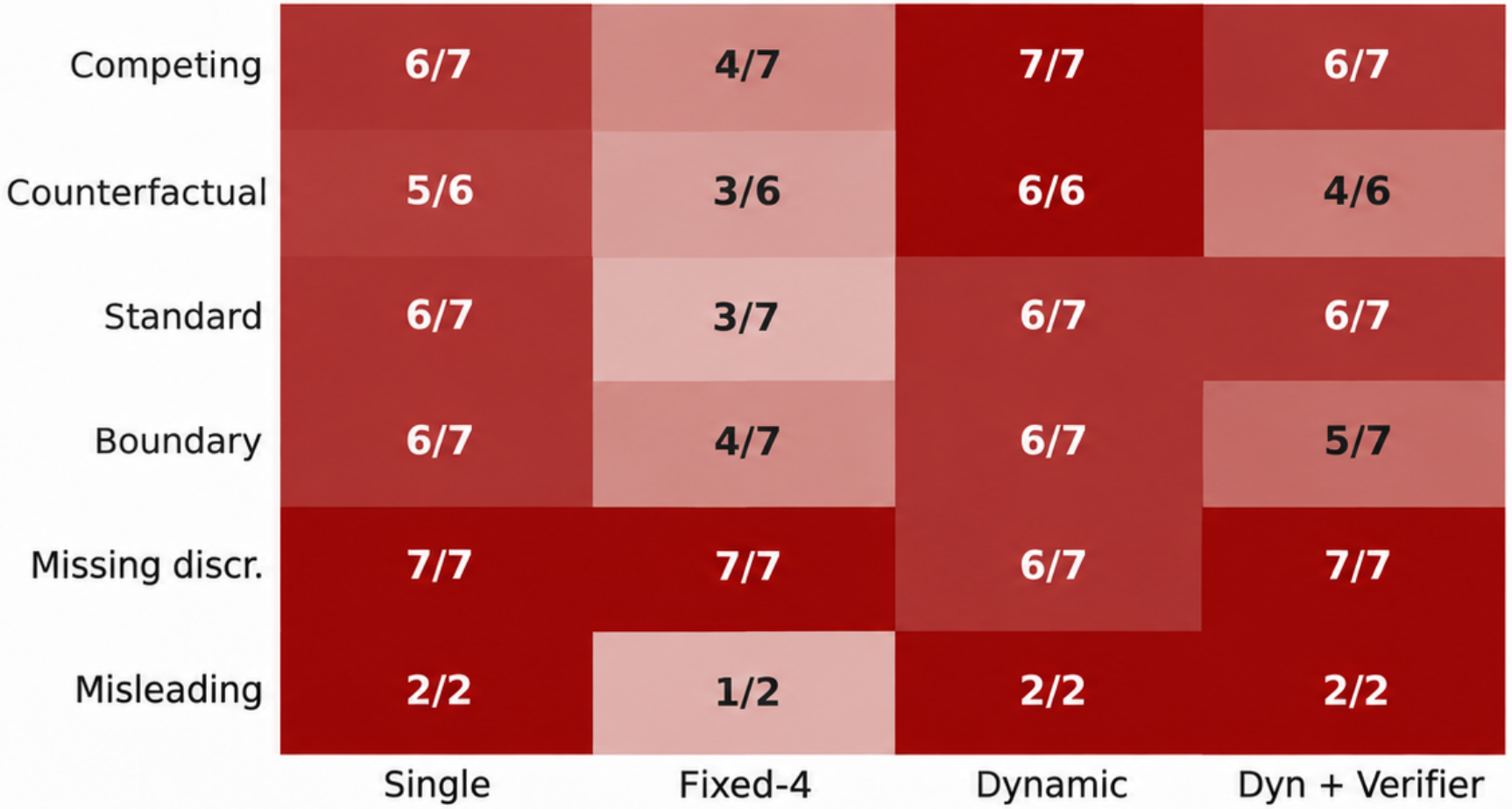
Dynamic denominator example

27 correct roots (completed identifiable) + **6 correct abstentions** (completed abstention) + **3 technical failures** (not completed) = **36 slots** (all cases)

Note: Quality successes = 33/36, not 27/27.

Results are on 36 synthetic development-visible cases with provisional labels, including technical failures.
Differences are descriptive, without a blind test, significance claim, human semantic audit or production generalization.

Backup: Paired outcomes and difficulty levers



Dynamic vs Single

3 wins, 2 losses, 31 ties.

Dynamic vs Fixed-4

12 wins, 1 loss, 23 ties.

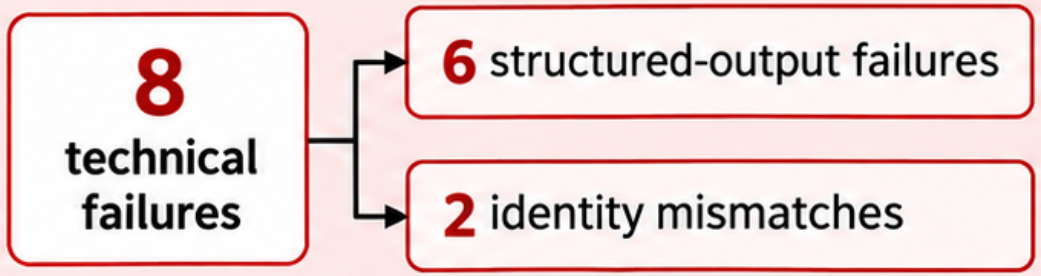
Dynamic vs verifier

5 wins, 2 losses, 29 ties.

Paired counts are descriptive; statistical significance was not established.

Backup: Technical failures and evidence exhaustion

Technical failures (execution errors)



- **8 technical failures:** 6 structured-output failures and 2 identity mismatches.
- **Dynamic:** 6 stops, all correct abstentions.
- **Verifier:** 11 stops, including 4 identifiable losses.
- **Single:** 10 stops, including 3 identifiable losses.

Evidence-exhaustion stops (guard-related coverage)

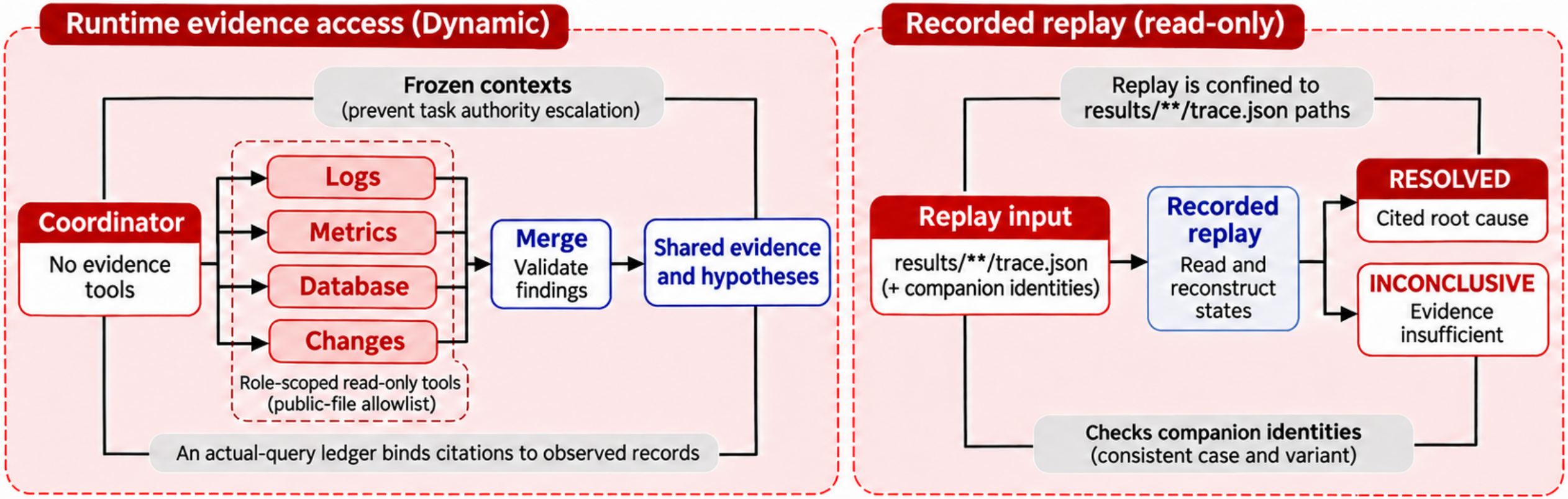
Evidence-exhaustion stops: **27/144** in the hardened 8-round protocol.

Variant	Stops (evidence exhaustion)	Identifiable losses (coverage)
Dynamic	6	0
Verifier (Dynamic with verifier)	11	4
Single	10	3
Fixed-4	0	0

- Stock 4-round budget differs from the 8-round benchmark protocol.
- Earlier replay-estimated token savings are not a newly measured execution result.

Do not attribute causation to architecture from failure counts. The analysis is descriptive and within the bounded evaluation.

Backup: Public evidence access and replay remain constrained



- Public-file allowlist and role-scoped read-only tools.
- Frozen contexts prevent task authority escalation.
- An actual-query ledger binds citations to observed records.
- Merge validates findings before shared publication.
- Replay is confined to results/**/trace.json paths and checks companion identities.

Replay does not execute the graph, tools or model provider. Private evaluator oracles are for scoring and not runtime evidence. No claim of complete security against every attack.

Backup: The demonstration can run offline

Validate data, run Dynamic with Mock, then replay the saved trace.

Reproducibility sequence (offline with Mock)

1 Validate data

```
python -m incident_lens.cli validate-data
```

2 Run Dynamic with Mock

```
python -m incident_lens.cli run --case incident_001 \
  --provider mock --variant dynamic \
  --cache-mode off --out results/demo
```

3 Replay the saved trace

```
python -m incident_lens.cli replay --trace results/demo/trace.json
```

Saved artifacts

- trace.json
- result.json
- report.md

- Outputs: trace.json, result.json and report.md.
- The dashboard and CLI use the same saved artifacts.
- A real-provider evaluation is separate from an offline Mock demonstration.

Start from the documented installed environment; real provider configuration is separate.